

Algorithms: Structural Graph Coverage Criteria

Meenakshi D'Souza

International Institute of Information Technology
Bangalore.

NPTEL

Graph coverage criteria: Overview

- We look at graphs as structures and consider the following two coverage criteria over graphs.
  - Structural coverage criteria.
  - Data flow coverage criteria.
- Later, we consider software artifacts (code, design elements and requirements) modelled as graphs and see how these coverage criteria apply to them.
- Focus of this lecture: Algorithms for structural coverage criteria over graphs.

NPTEL

Structural coverage criteria over graphs

Coverage criteria discussed in the previous lecture.

- Node/vertex coverage.
- Edge coverage.
- Edge pair coverage.
- Path coverage
  - Complete path coverage.
  - Prime path coverage.
  - Complete round trip coverage.
  - Simple round trip coverage.

Two entities: Test requirements and test paths

- There are two entities related to coverage crtieria:
  - Test requirement
  - Test case as a test path, if the test requirement is feasible.
- Algorithms need to discuss how to find and present the test requirements and how to obtain the test paths that meet the test requirements (if feasible).
- We don't consider undecidability results relating to checking for (in)feasible requirements.

NPTEL

Node, edge coverage: Algorithms

- Test requirements for node and edge coverage are already given with the graph.
  - TR for node coverage: The set of vertices/nodes.
  - TR for edge coverage: The set of edges and vertices/nodes.
- Test paths to achieve node and edge coverage can be obtained by simple modifications to BFS algorithm over graphs.

NPTEL

Edge-pair coverage: Algorithms

- TR for edge-pair coverage is all paths of length two in the given graph. Need to include paths that involve self-loops too.
- Test paths for edge-pair coverage:
  - At the basic level, the TR itself constitute the test paths. In fact, we can't do better than this for many graphs.
  - A simple algorithm that considers paths of length two by exploring adjacency list of a node and its successor once, for all nodes in the graph will do for the test paths.

NPTEL

Complete path coverage

- For graphs without loops, complete path coverage is achievable as it is basically the set of all paths in the graphs.
- The set of all paths is a finite set for graphs without loops.
- A simple modification of BFS algorithm will give both the TR and the test paths for complete path coverage.
- For graphs with loops (very common when we model code as graphs), complete path coverage is infeasible as a TR.

NPTEL

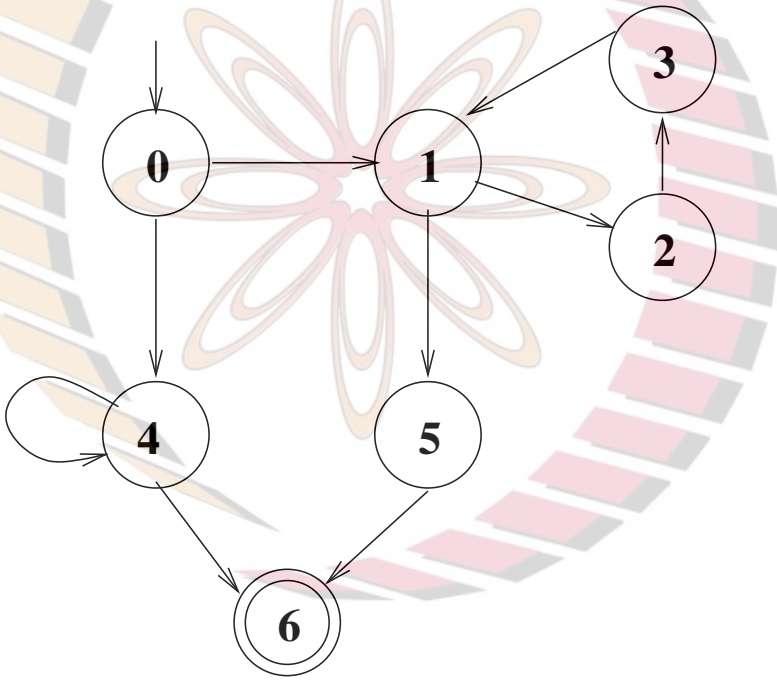
Prime path coverage: Test requirement

- Prime paths are *maximal* simple paths.
- Defining the set of prime paths to cover as a test requirement needs some work.
- We will first look at an algorithm to define and list the set of prime paths as a TR and then see how to get test paths to satisfy this TR.

NPTEL

Computing prime paths: An example

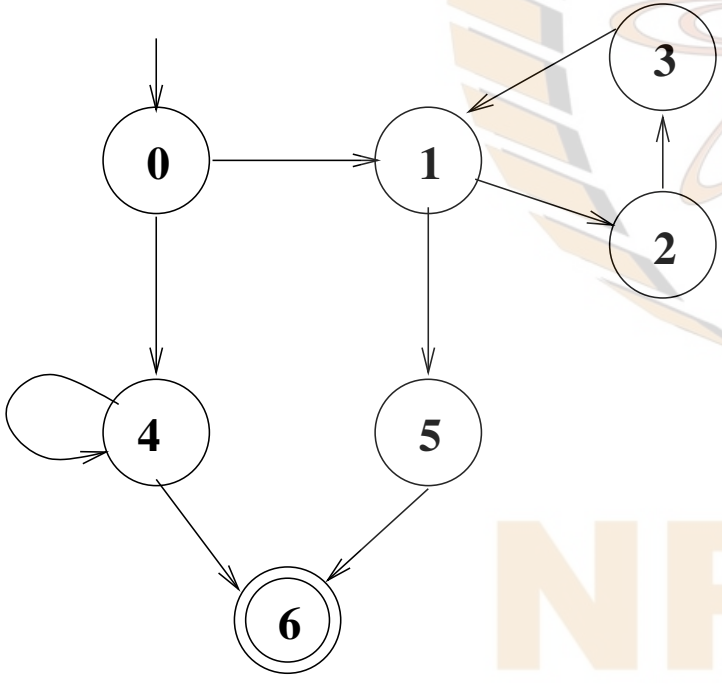
Consider the graph below:



- Our algorithm will enumerate all simple paths, in order of increasing length.
- We will then choose the prime paths from this list, as and when we enumerate the simple paths.

Computing prime paths: Example

Simple paths of length 0 (7 paths).

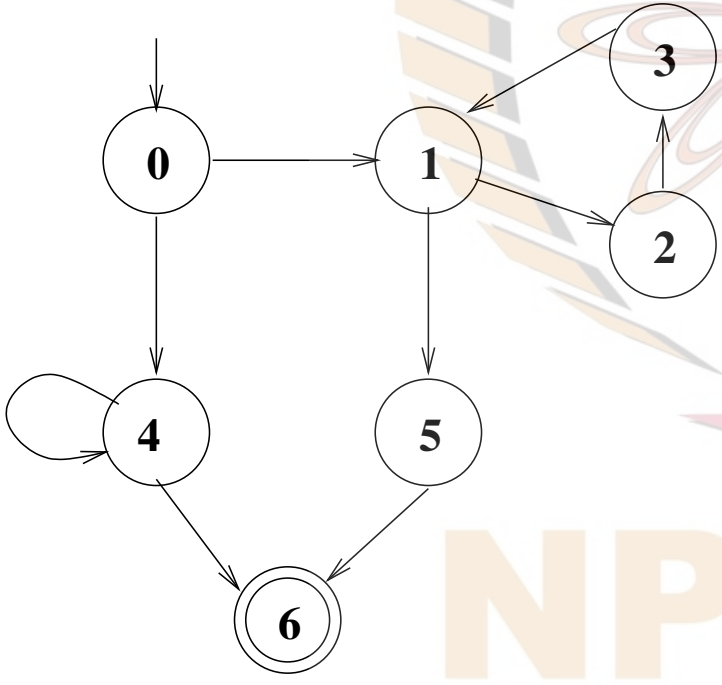


- Path [0]
- Path [1]
- Path [2]
- Path [3]
- Path [4]
- Path [5]
- Path [6]!

Exclamation mark (!) after path [6] implies [6] cannot be extended further. Note that 6 is a final node and has no out-going edges.

Computing prime paths: Example

Simple paths of length 1 (9 paths).



1. Paths [0,1], [0,4]
2. Paths [1,2], [1,5]
3. Path [2,3]
4. Path [3,1]
5. Paths [4,4]*, [4,6]!
6. Path [5,6]!

The asterisk (*) implies that the path cannot be extended further, it is already a simple cycle.

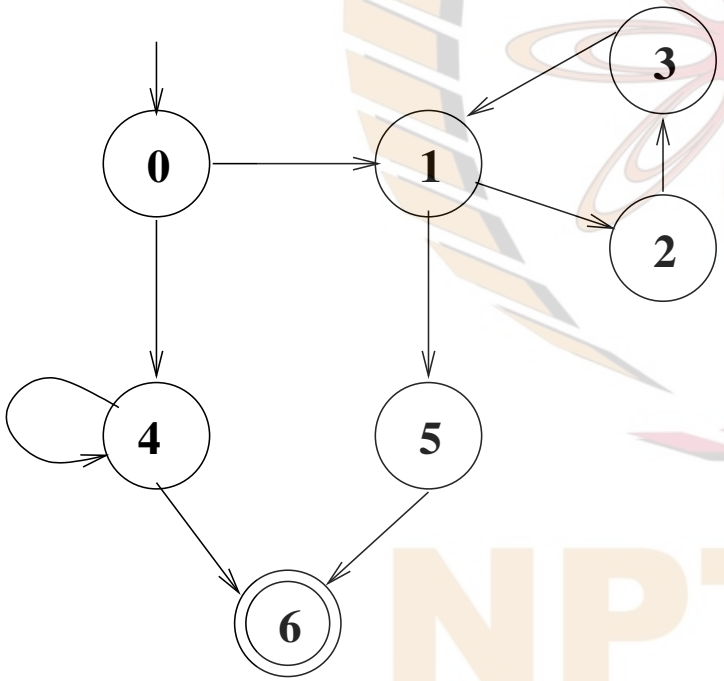
Computing prime paths: Algorithm

- To enumerate paths of length 2, consider each path of length 1 that is not a cycle (marked with a *).
- Extend the path of length 1 with every node that can be reached from the final node in the path, unless that node is already in the path and not the first node.
- Repeat the above till we reach paths of length $|V| - 1$, where V is the set of vertices of the given graph.

NPTEL

Computing prime paths: Example

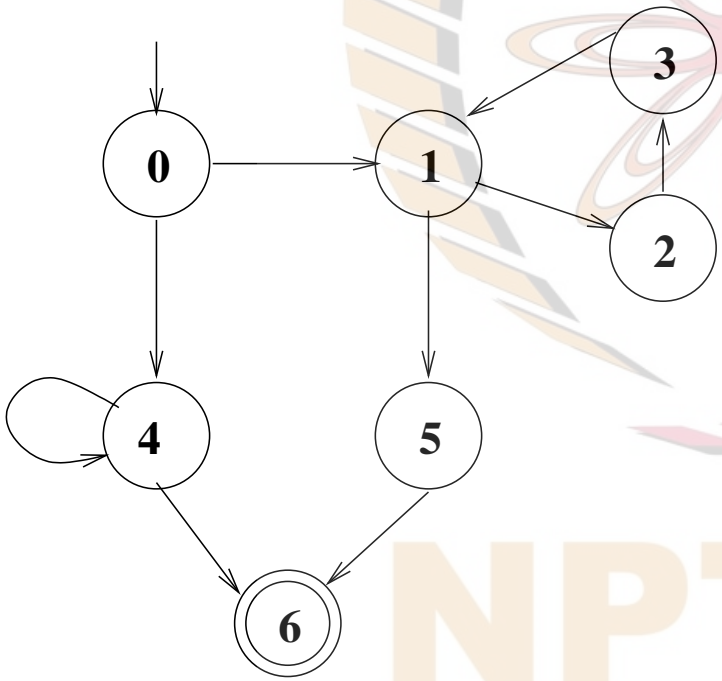
Simple paths of length 2 (8 paths).



- Paths [0,1,2], [0,1,5]
- Path [0,4,6]!
- Path [1,2,3]
- Path [1,5,6]!
- Path [2,3,1]
- Path [3,1,2]
- Path [3,1,5]

Computing prime paths: Example

Simple paths of length 3 (7 paths).



1. Path [0,1,2,3]!
2. Path [0,1,5,6]!
3. Path [1,2,3,1]*
4. Paths [2,3,1,2]*, [2,3,1,5]
5. Path [3,1,2,3]*
6. Path [3,1,5,6]!

Many paths of length 3 cannot be extended.

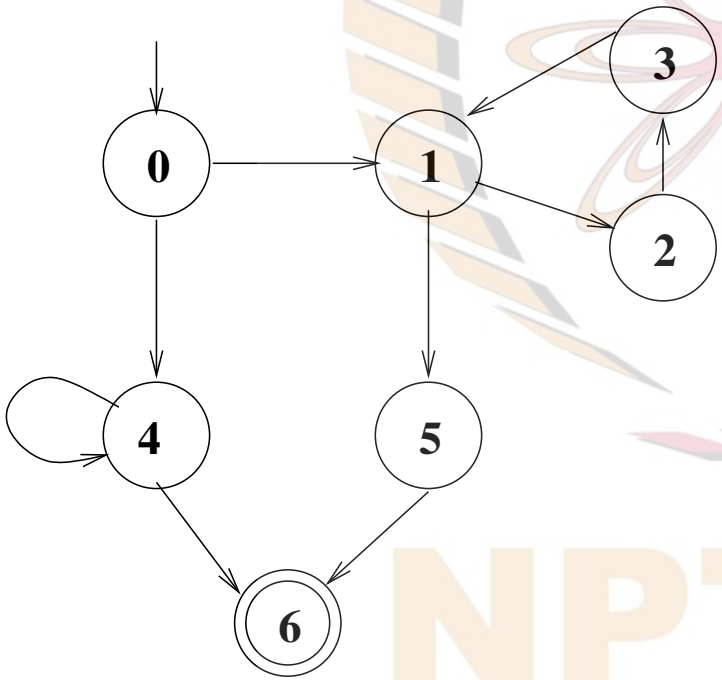
Computing prime paths: Example

- Only one path of length 4 exists: [2,3,1,5,6]!
- The above process is continued:
  Every simple path without a ! or a * can be extended.
- The process is guaranteed to terminate as the length of the longest prime path is the number of nodes.
- There are totally 32 simple paths in the example graph, only 8 of them are prime paths.

NPTEL

Computing prime paths: Example

There are 8 prime paths for this graph.



- Path [4,4]*
- Path [0,4,6]!
- Path [0,1,2,3]!
- Path [0,1,5,6]!
- Path [1,2,3,1]*
- Path [2,3,1,2]*
- Path [3,1,2,3]*
- Path [2,3,1,5,6]!

Test paths for prime path coverage

Outline of algorithm that will enumerate test paths for prime path coverage:

- Start with the longest prime paths and *extend* each of them to the initial and the final nodes in the graph.
- For the example, [2,3,1,5,6] is extended to [0,1,2,3,1,2,3,1,5,6]. This tours 4 prime paths— [0,1,2,3], [1,2,3,1], [2,3,1,2] and [3,1,2,3].
- The prime path [0,1,5,6] is itself a test path that satisfies the above condition.
- Continuing further, we get two more test paths [0,4,6] and [0,4,4,6].

NPTEL

Test paths for prime path coverage

- The complete set of test paths are
  - Test path [0,1,2,3,1,5,6]
  - Test path [0,1,2,3,1,2,3,1,5,6]
  - Test path [0,1,5,6]
  - Test path [0,4,6]
  - Test path [0,4,4,6]
- This is not the most *optimal* set of test paths. For e.g., the first test path is contained in the second one and hence can be omitted.

NPTEL

Test requirements and their test paths

- Given a TR for structural graph coverage, the problem of obtaining test paths for satisfying the TR mainly uses algorithms that extend BFS/DFS.
- They may or may not yield an *optimal* set of test paths that satisfy the TR.
- There are several notions of what an optimal set of test paths for a given TR is.
- Many algorithms that come up with optimal test paths for a given TR are NP-complete.

Symbolic execution based algorithms

- There are several *symbolic execution* based algorithms for obtaining tet paths to achieve coverage criteria.
- We will see symbolic execution later in the course and use it for path coverage in programs.

NPTEL

Some references

- There is a graph coverage webapp available in the webpage of the text book for this course:
  `https://cs.gmu.edu:8443/offutt/coverage/GraphCoverage`
- Useful webapp to try out and understand various structural graph coverage criteria.
- A good reference for test paths and test requirements on prime path coverage:
  Nan Li, Fei Li, and Jeff Offutt, Better Algorithms to Minimize the Cost of Test Paths, *IEEE 5th International Conference on Software Testing, Verification and Validation*, April 2012.